

Laporan Update Progress AuditChain Gateway

Blockchain-based Audit Trail Middleware

Periode: September 2026

Versi dokumen: 1.0

Nama Proyek	AuditChain Gateway
Jenis Sistem	Middleware Audit Trail berbasis Blockchain
Teknologi Utama	Hyperledger Fabric, Kafka, Debezium, Go, React
Disusun oleh	[isi nama]
Tanggal	[isi tanggal]

Daftar Isi

1	Ringkasan Update	2
1.1	Fokus Update September 2026	2
1.2	Nilai Utama Sistem	3
2	Status Progress Modul	4
2.1	Ringkasan Capaian	5
3	Update Fitur September 2026	6
3.1	Penambahan Mapping Actor per Operasi	6
3.1.1	Manfaat	6
3.1.2	Before dan After	6
3.2	Web Users Activity Monitoring	7
3.2.1	Perubahan Utama	7
3.2.2	Alur Data Web Users	8
3.3	Catatan Akurasi Web Users	8
4	Testing dan Validasi	9
4.1	Validasi Flow V1	9
4.2	Rencana Testing Actor Flow V2	9
4.2.1	Tabel Yang Digunakan	9
4.2.2	Target Validasi	9
4.3	Checklist Pass/Fail	10
4.4	Acceptance Criteria Tahap Berikutnya	11
5	Kendala dan Catatan Teknis	12
5.1	Kendala Implementasi Saat Ini	12
5.2	Batasan Sistem Yang Perlu Disampaikan	12
5.3	Catatan Keamanan Pra-Produksi	13
6	Roadmap Selanjutnya	14
6.1	Prioritas P0	14
6.2	Prioritas P1	14
6.3	Prioritas P2	14
6.4	Demo Akhir Yang Disarankan	15
7	Lampiran	16
7.1	Daftar Screenshot Yang Perlu Dimasukkan	16
7.2	Referensi File Internal	16
7.3	Catatan Penggunaan Template	17

1. Ringkasan Update

AuditChain Gateway adalah sistem middleware yang berfungsi sebagai API Gateway untuk menerima, memproses, dan mengunci log audit dari berbagai sistem klien secara immutable ke dalam jaringan Hyperledger Fabric. Sistem ini dirancang sebagai solusi multi-tenant berbasis SaaS yang dapat menerima data melalui Change Data Capture (CDC) menggunakan Kafka dan Debezium, serta melalui endpoint HTTP ingestion sebagai jalur alternatif.

Update September 2026 berfokus pada penguatan kemampuan audit operasional, terutama pada pelacakan actor, monitoring aktivitas user, kesiapan validasi end-to-end, serta penyusunan catatan teknis untuk tahap pra-produksi.

1.1 Fokus Update September 2026

1. Penyempurnaan konfigurasi actor tracking agar field actor dapat dibedakan berdasarkan operasi INSERT, UPDATE, dan DELETE.
2. Perubahan menu Web Users dari daftar user sederhana menjadi halaman activity monitoring.
3. Penyusunan skenario testing Actor Flow V2 untuk memastikan actor dari database client dapat terbaca dengan benar.
4. Identifikasi gap teknis pada ingestion, Kafka consumer, normalizer, keamanan, dan retention policy.
5. Penyusunan roadmap lanjutan menuju validasi core sistem dan kesiapan demo akhir.

Placeholder Gambar

Diagram arsitektur umum AuditChain Gateway
Ganti placeholder ini dengan diagram sistem dari database client, CDC agent, Kafka, Gateway, dashboard, dan Hyperledger Fabric.

Gambar 1.1: Diagram arsitektur umum AuditChain Gateway

1.2 Nilai Utama Sistem

Sistem ini tidak hanya menyimpan log, tetapi membangun evidence chain yang dapat diverifikasi. Setiap perubahan data diproses melalui hashing SHA3-256, dikelompokkan dengan Merkle Tree, lalu Merkle Root di-anchor ke blockchain. Dengan mekanisme tersebut, auditor dapat memeriksa apakah evidence yang diterima masih valid atau sudah berubah.

Pada update ini, peningkatan paling penting berada pada sisi attribution, yaitu kemampuan sistem untuk menjawab pertanyaan siapa yang melakukan perubahan data berdasarkan struktur database client yang berbeda-beda.

2. Status Progress Modul

Bagian ini merangkum status pengembangan per modul agar perkembangan sistem dapat dibaca secara lebih terukur.

Modul	Status	Keterangan
Autentikasi dan Multi-tenant	Berjalan	Mendukung API Key untuk client ke gateway, JWT Bearer untuk dashboard, serta mekanisme verify token untuk gateway ke agent client.
Client Registry	Berjalan	Admin dapat mendaftarkan tenant/client baru, menghasilkan API Key, serta mengatur konfigurasi koneksi dan mapping awal.
Kafka CDC Ingestion	Berjalan	Data perubahan database client dapat masuk melalui Kafka/Debezium. Reconciliation berjalan berkala untuk memantau topic dan event.
HTTP Ingestion	Sebagian	Endpoint ingestion HTTP tersedia sebagai jalur alternatif, tetapi beberapa bagian masih perlu integrasi penuh ke router utama.
Hashing Engine	Selesai awal	Log audit dihitung menggunakan SHA3-256 terhadap field kanonik agar integritas data dapat diperiksa ulang.
Merkle Aggregator	Berjalan	Log dapat dikelompokkan menjadi batch dan dibangun menjadi Merkle Tree dengan proof chain dari leaf ke root.
Fabric Anchoring	Berjalan	Merkle Root dapat dikirim ke jaringan Hyperledger Fabric sebagai anchor evidence.
Verification Engine	Berjalan	Mendukung verifikasi berlapis: database existence, local re-hash, agent source verification, dan blockchain proof.

Dashboard Audit Logs	Berjalan	Menampilkan audit logs, status integritas, filter, pencarian, pagination, dan modal verifikasi.
Web Users Activity Monitoring	Frontend selesai awal	Menu Web Users sudah membaca user CDC dan recent audit logs untuk menampilkan aktivitas terakhir user. Backend khusus activity masih menjadi rekomendasi lanjutan.
Actor Tracking	Dalam penyempurnaan	Form pendaftaran client sudah ditambah field Insert Actor, Update Actor, Delete Actor, dan Placeholder Actor. Integrasi penuh ke consumer dan normalizer masih perlu dilanjutkan.
Retention dan Storage Monitoring	Belum tersedia	Perlu desain policy, monitoring storage, legal hold, dry-run purge, dan archive workflow.
Security Alert Engine	Belum tersedia	Belum ada rule engine untuk mass delete, unknown actor, off-hours activity, atau pipeline outage.

2.1 Ringkasan Capaian

Secara umum, core pipeline AuditChain Gateway sudah terbentuk dari ingestion sampai verifikasi. Fokus pekerjaan berikutnya bukan lagi membuat komponen dasar dari nol, tetapi memperkuat konsistensi data, validasi end-to-end, observability, dan batas keamanan sistem.

3. Update Fitur September 2026

3.1 Penambahan Mapping Actor per Operasi

Pada update ini, proses registrasi client disempurnakan dengan menambahkan field pemetaan actor yang lebih granular. Sebelumnya sistem cenderung bergantung pada satu field actor umum. Dengan perubahan baru, admin dapat menentukan sumber actor berbeda untuk operasi INSERT, UPDATE, dan DELETE.

Field	Fungsi
Insert Actor	Kolom yang digunakan untuk mengambil actor ketika terjadi operasi INSERT.
Update Actor	Kolom yang digunakan untuk mengambil actor ketika terjadi operasi UPDATE.
Delete Actor	Kolom yang digunakan untuk mengambil actor ketika terjadi operasi DELETE.
Placeholder Actor	Nilai contoh atau petunjuk pengisian, misalnya <code>created_by</code> , <code>updated_by</code> , <code>deleted_by</code> , atau <code>user_id</code> .

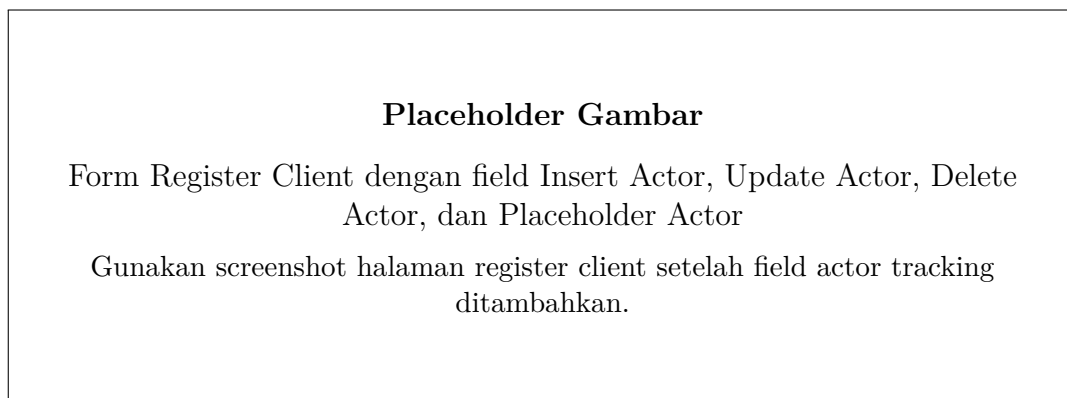
3.1.1 Manfaat

1. Sistem dapat mengikuti struktur database client yang berbeda-beda.
2. Actor pada audit log menjadi lebih akurat karena tidak dipaksa memakai satu nama kolom umum.
3. Admin lebih mudah melakukan onboarding client baru karena form memberi arahan field mapping.
4. Kasus actor **Unknown** dapat dikurangi selama database client memang menyimpan informasi actor.

3.1.2 Before dan After

Sebelum Update	Sesudah Update
----------------	----------------

Sistem memakai satu actor field umum atau fallback.	Sistem dapat memakai actor field berbeda untuk INSERT, UPDATE, dan DELETE.
Database client dengan kolom unik seperti <code>petugas_input</code> dapat menghasilkan actor <code>Unknown</code> .	Admin dapat mengisi <code>Insert Actor = petugas_input</code> , sehingga event baru dapat membaca actor dengan benar.
Konfigurasi actor kurang eksplisit untuk tiap jenis operasi.	Mapping actor lebih granular dan lebih mudah dijelaskan saat demo.



Gambar 3.1: Form Register Client dengan field Insert Actor, Update Actor, Delete Actor, dan Placeholder Actor

3.2 Web Users Activity Monitoring

Menu Web Users diperbarui dari sekadar daftar user menjadi halaman monitoring aktivitas user. Frontend mengambil data user dari endpoint dashboard user, lalu mengambil recent audit logs dari endpoint dashboard logs. Data tersebut kemudian dicocokkan berdasarkan actor, username, email, atau full name.

3.2.1 Perubahan Utama

1. Tabel Web Users menampilkan aktivitas terakhir user.
2. Kolom yang ditampilkan mencakup User, Username, Email, Last Activity, Action Time, dan Status.
3. Row user dapat diklik untuk membuka drawer detail.
4. Drawer detail menampilkan identity, latest activity, source, dan raw attributes.
5. Export CSV membawa data aktivitas, bukan hanya data identitas user.
6. Status aktivitas meliputi Success, Pending, Needs Review, dan No Activity.

3.2.2 Alur Data Web Users

Frontend Web Users

- > GET /api/dashboard/my-users
- > GET /api/dashboard/logs?page=1&page_size=200&sort_order=desc
- > Normalize users
- > Normalize recent audit logs
- > Match actor to username/email/full_name
- > Attach latest activity
- > Render table and drawer

Placeholder Gambar

Screenshot Web Users Activity Monitoring

Gunakan screenshot tabel Web Users yang sudah menampilkan Last Activity, Action Time, dan Status.

Gambar 3.2: Screenshot Web Users Activity Monitoring

Placeholder Gambar

Drawer Detail Web Users

Gunakan screenshot drawer kanan yang menampilkan detail identity dan latest activity.

Gambar 3.3: Drawer Detail Web Users

3.3 Catatan Akurasi Web Users

Matching activity saat ini masih dilakukan di frontend dengan pendekatan heuristic. Untuk tahap produksi, endpoint backend khusus lebih disarankan agar pencocokan user dan audit log dilakukan secara presisi di sisi server.

Endpoint yang direkomendasikan:

```
GET /api/dashboard/my-users/activity
```

Dengan endpoint tersebut, backend dapat melakukan query, actor mapping, pagination, dan status normalization secara lebih konsisten.

4. Testing dan Validasi

4.1 Validasi Flow V1

Testing V1 membuktikan bahwa pipeline utama sudah dapat berjalan dari installer, koneksi agent, Debezium, Kafka, Gateway, Dashboard, sampai proses verify. Namun, pada beberapa skenario awal actor masih muncul sebagai **Unknown**. Hal tersebut tidak langsung berarti pipeline gagal, karena tabel dummy yang digunakan belum selalu memiliki konteks actor seperti `created_by`, `updated_by`, `deleted_by`, atau `user_id`.

4.2 Rencana Testing Actor Flow V2

Testing V2 disiapkan untuk memvalidasi actor resolution dari database client sampai dashboard. Skenario ini menggunakan database yang actor-aware, yaitu database yang memang menyediakan kolom pelaku perubahan.

4.2.1 Tabel Yang Digunakan

1. `public.app_users`: sumber identitas user aplikasi client.
2. `public.orders`: tabel bisnis yang memiliki kolom actor.

4.2.2 Target Validasi

Skenario	Expected Result
INSERT order	Actor terbaca dari field insert actor atau <code>created_by</code> .
UPDATE order	Actor terbaca dari field update actor atau <code>updated_by</code> .
DELETE atau soft delete order	Actor terbaca dari field delete actor atau <code>deleted_by</code> .
User source masuk dari CDC	Data user muncul di Web Users atau user tracking.
Actor berupa UUID	Sistem mencoba resolve UUID ke email atau full name dari <code>client_users</code> .

Verify log Hash dan verification tetap berhasil selama data tidak berubah.

4.3 Checklist Pass/Fail

Checklist Validasi

- PASS jika `client_users` berisi user dari `public.app_users`.
- PASS jika audit logs untuk tabel `orders` tidak lagi menampilkan actor `Unknown` ketika mapping benar.
- PASS jika actor UUID dapat di-resolve menjadi email atau full name.
- PASS jika verify hash dan blockchain proof tetap sukses.
- FAIL jika field actor ada nilainya tetapi audit log tetap `Unknown`.
- FAIL jika event CDC masuk tetapi tidak muncul di dashboard.

Placeholder Gambar

Screenshot Audit Logs dengan Actor Valid

Gunakan screenshot audit logs setelah event INSERT, UPDATE, atau DELETE menampilkan actor sesuai mapping.

Gambar 4.1: Screenshot Audit Logs dengan Actor Valid

Placeholder Gambar

Screenshot Verification Modal

Gunakan screenshot hasil verifikasi log yang menunjukkan status valid.

Gambar 4.2: Screenshot Verification Modal

4.4 Acceptance Criteria Tahap Berikutnya

Core sistem dapat dianggap siap untuk demo lanjutan jika memenuhi kriteria berikut:

1. Event INSERT, UPDATE, dan DELETE dari database client muncul di dashboard.
2. Actor tidak lagi **Unknown** jika field mapping sudah benar dan data actor tersedia di payload CDC.
3. Hash verification berhasil untuk log yang tidak berubah.
4. Merkle proof cocok dengan Merkle root.
5. Dashboard dapat menampilkan status valid, tampered, pending, dan unreachable.
6. Admin dapat mendaftarkan client dan mengatur mapping actor tanpa mengubah kode utama gateway.

5. Kendala dan Catatan Teknis

5.1 Kendala Implementasi Saat Ini

1. Beberapa endpoint ingestion HTTP pada modul `internal/modules/ingestion` masih tersedia, tetapi belum seluruhnya terhubung penuh ke router utama.
2. Consumer Kafka dan normalizer masih perlu disesuaikan agar membaca mapping actor per operasi secara konsisten.
3. Fallback actor harus tetap berjalan ketika field khusus INSERT, UPDATE, atau DELETE kosong.
4. Web Users Activity Monitoring saat ini masih melakukan matching activity di frontend sehingga akurasi bergantung pada recent logs yang diambil.
5. Belum tersedia endpoint backend khusus untuk user activity.
6. Belum tersedia retention policy, archive workflow, scheduled purge, atau storage quota.
7. Belum tersedia rule-based security alert untuk mass delete, unknown actor, atau pipeline outage.

5.2 Batasan Sistem Yang Perlu Disampaikan

AuditChain Gateway dapat membuktikan integritas evidence yang sudah diterima oleh sistem. Namun, sistem tidak boleh diklaim sebagai alat yang dapat membuktikan semua kejadian secara absolut.

Area	Batasan
Actor attribution	Actor menunjukkan identitas yang dikirim atau diturunkan dari sistem sumber. Jika akun user dicuri, sistem belum tentu mengetahui manusia asli di balik akun tersebut.
Completeness	Hash membuktikan data yang diterima tidak berubah, tetapi tidak otomatis membuktikan tidak ada event yang sengaja dicegah agar tidak pernah masuk.

Pipeline outage	Jika connector CDC mati sebelum perubahan data dibaca, sistem membutuhkan heartbeat dan lag monitoring untuk mendeteksi gap.
Blockchain anchoring	Blockchain menyimpan anchor Merkle Root. Raw evidence dan Merkle proof tetap perlu dikelola dengan retention dan archive yang benar.
Database client	Actor hanya dapat dibaca jika database client atau payload CDC memang menyediakan informasi pelaku perubahan.

5.3 Catatan Keamanan Pra-Produksi

Beberapa aspek keamanan yang perlu direview sebelum sistem digunakan secara lebih luas:

1. CORS whitelist.
2. Validasi JWT secret.
3. Rotasi dan penyimpanan API Key.
4. Pembatasan role admin, auditor, dan tenant.
5. Rate limiting endpoint publik.
6. Logging akses dashboard dan perubahan konfigurasi client.
7. Validasi input untuk field mapping actor.

6. Roadmap Selanjutnya

6.1 Prioritas P0

1. Integrasi penuh actor mapping per operasi ke Kafka consumer dan normalizer.
2. Penyimpanan actor raw dan actor display secara terpisah agar evidence forensik tidak kehilangan nilai asli dari sumber data.
3. Endpoint backend khusus `/api/dashboard/my-users/activity`.
4. Testing end-to-end Actor Flow V2 menggunakan tabel actor-aware.
5. Pipeline health monitoring, termasuk heartbeat, connector status, last event time, consumer lag, dan replication slot lag.

6.2 Prioritas P1

1. Storage observability untuk audit database, Kafka, Docker log, dan WAL database client.
2. Configurable retention policy per client.
3. Dry-run purge untuk evidence yang sudah eligible.
4. Rule-based security alert untuk mass delete/update, unknown actor, off-hours activity, dan connector down.
5. Monitored table reconciliation untuk menambah atau menonaktifkan tabel tanpa full reinstall.

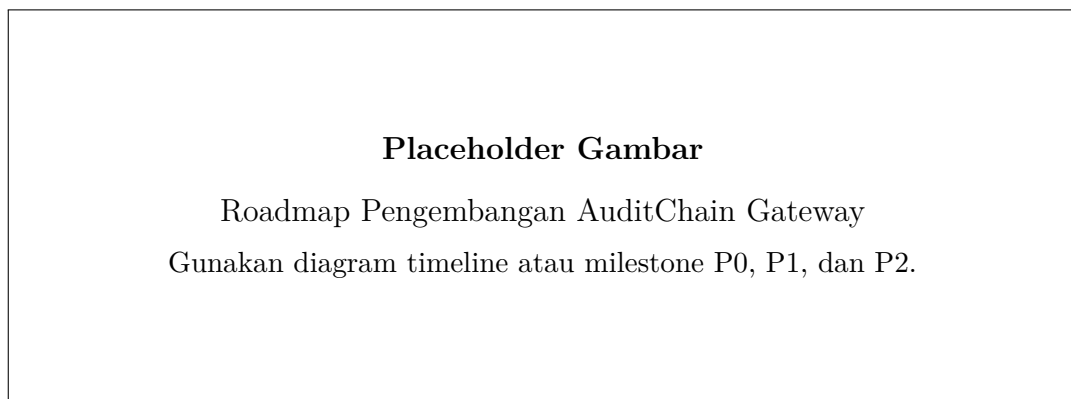
6.3 Prioritas P2

1. Archive evidence ke object storage atau media penyimpanan WORM.
2. Approval workflow dua pihak untuk perubahan konfigurasi sensitif.
3. Webhook atau email alert.
4. Correlation dengan login/auth event, IP, device, dan MFA.
5. Sequence manifest untuk memperkuat pembuktian completeness per interval.

6.4 Demo Akhir Yang Disarankan

Skenario demo yang kuat untuk tahap berikutnya:

1. Tenant admin mengaktifkan monitoring tabel `public.orders`.
2. Aplikasi client login sebagai user tertentu.
3. Backend client mengisi actor dari session.
4. User melakukan INSERT, UPDATE, dan DELETE.
5. Dashboard menampilkan actor, action, resource, timestamp, dan status integrity.
6. Auditor menjalankan verification terhadap salah satu log.
7. Sistem menampilkan hash valid, Merkle proof valid, dan anchor blockchain sesuai.
8. Simulasi actor tidak tersedia menghasilkan status `Unknown` secara jujur.
9. Simulasi mass delete menghasilkan security alert pada roadmap lanjutan.



Gambar 6.1: Roadmap Pengembangan AuditChain Gateway

7. Lampiran

7.1 Daftar Screenshot Yang Perlu Dimasukkan

Berikut daftar gambar yang disarankan untuk mengganti placeholder pada dokumen ini:

1. Diagram arsitektur umum AuditChain Gateway.
2. Screenshot dashboard overview.
3. Screenshot audit logs dengan actor valid.
4. Screenshot form Register Client dengan field actor per operasi.
5. Screenshot Web Users Activity Monitoring.
6. Screenshot drawer detail Web Users.
7. Screenshot verification modal.
8. Screenshot Kafka/Debezium connector running.
9. Screenshot Swagger atau dokumentasi API.

7.2 Referensi File Internal

Dokumen update ini disusun berdasarkan kondisi laporan progress sebelumnya dan catatan internal berikut:

- PDF progress September pada folder dokumentasi-awal-hingga-akhir
- penjelasan-keadaan-sistem-now/update_pdf.md
- penjelasan-keadaan-sistem-now/penjelasan-tracking-actor-v2.md
- dokumentasi-testing/testing-v2-actor-flow.md
- plans/plan-v19-core-completion-retention-actor-table-security.md

7.3 Catatan Penggunaan Template

Untuk mengganti placeholder gambar, letakkan file gambar pada folder **images/**, lalu ubah perintah `\placeholderimage` menjadi:

```
\begin{figure}[H]
  \centering
  \includegraphics[width=0.9\textwidth]{images/nama-file.png}
  \caption{Judul gambar}
\end{figure}
```

Jika hanya ingin membuat PDF awal tanpa gambar asli, dokumen ini tetap dapat di-build karena semua gambar menggunakan placeholder.